

Informe de Práctica: Z80 Assembly Project

Microprocesadores — Ciclo Escolar 27-II

Autor: Israel Juárez Contreras

Repositorio: Microprocesadores_27II

Entorno: Z80 Workbench & GitHub Desktop (macOS)

1 Resumen

En este documento se describe el desarrollo e implementación de un programa en lenguaje ensamblador para el microprocesador Z80. Se detalla la solución de los errores de sintaxis al compilar literales hexadecimales (como el uso de `0f800h`), la estructura de directorios del proyecto y el flujo de control de versiones mediante Git y GitHub Desktop.

2 Introducción

El estudio de los sistemas basados en microprocesadores requiere comprender tanto la arquitectura a nivel de hardware como la programación de bajo nivel. El microprocesador Z80 es un estándar clásico para el aprendizaje de control de buses, gestión de memoria y manipulación de registros. El presente trabajo plasma la configuración inicial de un sistema mínimo simulado y la subida formal del código al repositorio remoto institucional.

3 Funcionamiento del Sistema

3.1 Microprocesador

El sistema está basado en la arquitectura del microprocesador Zilog Z80, el cual opera con un bus de direcciones de 16 bits (permitiendo direccionamiento de hasta 64 KB) y un bus de datos de 8 bits. Cuenta con un conjunto de registros de propósito general y especiales (A, HL, BC, DE), los cuales se emplean en este diseño para apuntar a posiciones específicas de memoria y realizar operaciones incrementales de control.

3.2 Memorias

Se configuró un mapa de memoria básico simulado a partir de la dirección `0000h` para el almacenamiento del código de operación, y se destinó la sección superior de memoria a partir de la dirección `0f800h` para operaciones de escritura y almacenamiento temporal de datos generados por el programa ejecutado.

3.3 Interfaz para periféricos

Para este nivel de la práctica, la interacción se centra en la ejecución secuencial en memoria interna y la simulación por medio de entornos virtuales (*Z80 Workbench*), preparando el terreno para futuras ampliaciones mediante puertos de entrada y salida mapeados en E/S.

4 Diseño del Sistema

4.1 Diagrama electrónico

El proyecto se organizó bajo una estructura modular de carpetas en el repositorio, donde la carpeta `Hardware/` almacena los archivos esquemáticos correspondientes (diseñados en herramientas como KiCad) que definen las conexiones físicas teóricas del bus de direcciones, datos y control hacia los bancos de memoria y circuitería de apoyo.

4.2 Código Fuente

El software implementado y validado en la carpeta `software/programa.asm` es el siguiente:

```
; Configuración inicial para Z80
ORG 0000h

INICIO:
    LD HL, 0f800h    ; Corrección de sintaxis hexadecimal
    LD A, 00h

CICLO:
    INC A
    LD (HL), A
    JP CICLO

END
```

Listing 1: Código ensamblador corregido (programa.asm)

5 Análisis de los Resultados

Durante el proceso de compilación inicial, el simulador arrojó errores debido a la ambigüedad en la lectura de constantes hexadecimales sin prefijo numérico. Al corregir la instrucción asignando `0f800h`, el compilador interpretó correctamente el operando sin confusiones con etiquetas o símbolos. Asimismo, se logró resolver el versionado local excluyendo archivos temporales del sistema operativo (`.DS_Store`) para asegurar una subida limpia al repositorio remoto mediante GitHub Desktop.

6 Conclusión

La práctica permitió consolidar los conocimientos teóricos sobre la arquitectura interna del Z80 y su lenguaje ensamblador. Se comprobó la importancia de cuidar la sintaxis estricta en los entornos de desarrollo de bajo nivel y se afianzaron las habilidades en el uso de herramientas modernas de control de versiones para la entrega formal de proyectos de ingeniería.

7 Referencias

- Zilog Inc. *Z80 CPU Technical Manual*. Manual de referencia de arquitectura y conjunto de instrucciones.
- Apuntes y guías de laboratorio de la materia de Microprocesadores (Ciclo 27-II).

- Documentación oficial de GitHub Desktop y gestión de repositorios Git.